

AI PESTICIDE DISPENSAL AGENT

Multi-Step Vision & Prompt Workflow for Precision Agriculture & Crop Health

STUDENT: GOWTHAM KAKKE	REGISTER NO.: 99230040028
ASSESSMENT: Mini Assessment — Prompt Workflow Design	

01 | Concept

Purpose: The AI Pesticide Dispensal Agent processes live visual feeds from field cameras and agricultural drones, classifies crop infection levels, retrieves precise chemical dosage guidelines, drafts hardware control commands for the sprinkler system, and audits for environmental safety.

Problem: Blanket pesticide spraying leads to severe chemical runoff, ecological damage, and wasted resources. A multi-step multimodal AI pipeline standardizes crop evaluation, ensuring intelligent, targeted pesticide application determined by the actual infection level of the plant.

02 | Objective

The workflow uses sequential prompts and vision models to analyze plant health, categorize pest/disease severity, query agricultural knowledge bases, generate precise sprinkler hardware commands, and evaluate environmental constraints before spraying.

03 | Workflow Overview

1. Ingest	2. Classify	3. Retrieve	4. Draft Cmd	5. Audit
-----------	-------------	-------------	--------------	----------

04 | Workflow Steps

- 1. Image Parsing & Feature Extraction: Analyze visual feed from field sensors to identify the plant species and visible anomalies (e.g., discoloration, pest presence). [Output: Structured Crop Data]
- 2. Severity Classification: Categorize the infection level into Critical, Moderate, Mild, or Healthy. [Output: Infection Severity + Target Zone]
- 3. Knowledge Base Matching: Retrieve pertinent agronomic guidelines, chemical dosage limits, and active weather constraints (wind/rain). [Output: Agricultural Context]
- 4. Command Generation: Draft the precise hardware activation JSON, specifying the nozzle ID, spray duration, and chemical mixture ratio. [Output: Draft Sprinkler Command]

- 5. Environmental Audit: Verify weather conditions and toxicity limits. If wind speeds exceed 15 km/h (risk of drift), abort spraying and alert the farmer. [Output: Final Action / Execution Log]

05 | Prompt Workflow

Step 1 — Parse Field Camera Feed

Analyze the incoming drone/camera image. Identify the crop type, detect any visible insects, fungal growths, or leaf discoloration, and calculate the affected surface area percentage.

Step 2 — Classify Infection Level

Based on the extracted features, assign an infection severity score (0-100) and categorize the health status of the target sector.

Step 3 — Retrieve Dosage Guidelines

Match the identified pest/disease and crop type against the registered pesticide database. Select the optimal chemical and calculate the base dosage required.

Step 4 — Draft Hardware Command

Using the required dosage and current weather telemetry, format a JSON command for the smart sprinkler controller, specifying [nozzle_id], [flow_rate], and [duration_seconds].

Step 5 — Safety & Environmental Audit

Review the drafted command against live weather data (wind speed, precipitation forecast). If conditions risk chemical drift or runoff, override the command, log a "Weather Hold", and notify the system admin.

06 | Prompt Chaining Architecture

Step 1 produces visual metadata; Step 2 determines the severity; Step 3 gathers chemical context; Step 4 crafts the machine instructions; and Step 5 audits environmental safety thresholds.

STAGE	INPUT / CONTEXT	OUTPUT
1	Raw Field Image	Crop Species & Anomaly Data
2	Step 1 Anomaly Data	Infection Severity Category
3	Step 2 Category + Agri DB	Recommended Pesticide & Base Dose
4	Step 3 Rules + Weather API	JSON Sprinkler Hardware Command
5	Step 4 Command + Wind/Rain Data	Approved Execution / Weather Abort

07 | Demonstration Input

Example Scenario: A sector 4 field camera captures severe aphid infestation on tomato plants (40% leaf coverage). Current local weather is clear, temperature 24°C, with a mild wind speed of 5 km/h.

08 | Expected Final Output & Workflow Result

An automated risk classification of "Critical Infestation", retrieval of a Neem-based organic pesticide protocol, generation of a 30ml/sec targeted spray command for Nozzle 4A, and an approved execution log.

10 | Sample Support Execution Plan

PHASE	ACTION FOCUS	OUTPUT / METRIC
Phase 1	Anomaly Detection	Aphids detected → Vision model flags infestation
Phase 2	Severity Assessment	40% coverage confirmed → Critical protocol initiated
Phase 3	Command Drafting	Telemetry captured → JSON spray command prepared
Phase 4	Environmental Audit	Wind speed acceptable → Routed to IoT Sprinkler Controller
Phase 5	Resolution Track	Sprinkler activated → Crop treated and logged in dashboard

11 | Key Benefits

- ✓Reduces chemical runoff and promotes sustainable agriculture
- ✓Automates hardware integration for targeted, plant-level treatment
- ✓Prevents crop loss through real-time disease monitoring
- ✓Ensures compliance with environmental safety regulations
- ✓Highly scalable architecture utilizing vision models and IoT telemetry

12 | Discussion Questions & Answers

1. What is the advantage of using {user_input} instead of hard-coding input?

Templating separates the prompt's fixed structure from the dynamic variables. By using {image_features} and {weather_data}, the intelligent sprinkler system can process thousands of unique sensor feeds across the farm dynamically, without a developer needing to hard-code instructions for every individual plant's current state.

2. When would .format() be enough?

Python's `.format()` is perfectly sufficient for simple, linear steps, such as injecting the identified `{crop_type}` and `{pest_name}` into a static query sent to the agricultural knowledge base. If no complex conditional logic is required, it keeps the code lightweight.

3. When might Jinja2 be useful?

Jinja2 is essential when generating the hardware command. You can use Jinja `{% if %}` blocks to add a 'reduce spray pressure' parameter only if the weather API reports high winds, or loop through multiple infected coordinates to generate a batched spray sequence. It allows the prompt to adapt structurally to the environment.

4. Why should prompts be versioned?

Agricultural guidelines and vision model capabilities change over time. If a newly updated prompt inadvertently causes the system to hallucinate higher chemical dosages, versioning provides an audit trail and allows for an immediate rollback to prevent catastrophic chemical burn to the crops.

5. What is a prompt regression?

A prompt regression occurs when an adjustment (e.g., tweaking the instructions to better identify subtle fungal infections) unintentionally breaks the model's existing ability to accurately classify common pests like aphids. It shows how fragile prompt phrasing can be across diverse tasks.

6. Why use the same evaluation set when comparing prompt versions?

Using a static 'golden dataset' of crop images ensures an apples-to-apples comparison. If you change the prompt and the test images simultaneously, you cannot isolate the variables—any change in diagnostic accuracy could be due to the new prompt, or simply because the new images had better lighting.